

UNIVERSIDADE ESTADUAL DA PARAÍBA
CAMPUS I – EDVALDO DE SOUZA DO Ó
CENTRO DE CIÊNCIA E TECNOLOGIA
DEPARTAMENTO DE COMPUTAÇÃO
CURSO DE CIÊNCIA DA COMPUTAÇÃO

SUELLE FERREIRA MACIEL

FILAS DE MENSAGENS POSIX

Relatório da atividade 4 de Sistemas Operacionais

CAMPINA GRANDE
2026

1 INTRODUÇÃO

Esta atividade utiliza filas de mensagens POSIX para estabelecer a comunicação entre dois processos independentes. Nesse mecanismo, as mensagens são depositadas em filas mantidas pelo núcleo do sistema operacional, das quais podem ser retiradas por outros processos (MAZIERO, 2019). O processo A sorteia pares de números inteiros n1 e n2, entre 0 e 99, e os envia ao processo B. O processo B calcula a soma, aguarda 1 segundo e devolve o resultado ao processo A, que imprime a operação no formato n1 + n2 = soma.

Além da implementação da solução, o roteiro solicita a avaliação de seu comportamento com um e com dois processos de cada tipo e a proposição de uma forma adequada de resolver os problemas eventualmente observados.

2 AMBIENTE DE EXECUÇÃO

Como o computador utilizado executa o sistema operacional Windows, a atividade foi desenvolvida no WSL2 (Windows Subsystem for Linux), com a distribuição Ubuntu 24.04. As chamadas mq_open, mq_send e mq_receive não estão disponíveis no Windows, mas o WSL2 executa um núcleo Linux real, de modo que as filas POSIX funcionam normalmente, sem a necessidade de uma máquina virtual. A preparação do ambiente seguiu as etapas abaixo, conforme a documentação oficial do WSL (MICROSOFT, 2025).

1. **Instalação do WSL e do Ubuntu**, por meio do PowerShell executado como administrador. Após a reinicialização do computador, foram criados o usuário e a senha do Linux:

   ```
   wsl --install -d Ubuntu-24.04
   ```

2. **Instalação do compilador**, no terminal do Ubuntu. A primeira tentativa falhou porque a lista de pacotes estava desatualizada, o que foi resolvido com sua atualização prévia:

   ```
   sudo apt update && sudo apt install build-essential
   ```

3. **Acesso à pasta do projeto.** No WSL, o disco C: do Windows é acessado pelo caminho /mnt/c:

```
cd /mnt/c/Users/suell/Documents/UEPB/2026.2/
  SistemasOperacionais/Atividade04
```

4. **Compilação**, com a ligação da biblioteca de tempo real (-lrt), que contém as funções mq_* (LINUX MAN-PAGES PROJECT, 2026):

```
gcc processoA.c -o A -lrt
gcc processoB.c -o B -lrt
gcc limpa.c -o limpa -lrt
```

5. **Execução.** Cada processo (./A ou ./B) foi executado em um terminal próprio do Ubuntu, dispostos lado a lado no Terminal do Windows, e encerrado com Ctrl+C. Antes de cada cenário, executou-se o programa ./limpa, que remove as filas com mq_unlink.

3 SOLUÇÃO

3.1 QUANTIDADE DE FILAS NECESSÁRIAS (ITEM 1)

São necessárias **duas filas**, uma para cada sentido da comunicação, conforme o Quadro 1.

Quadro 1 – Filas de mensagens utilizadas na solução

Fila	**Sentido**	**Conteúdo**
/fila-suelle-pedidos	A → B	struct pedidoT, com n1 e n2
/fila-suelle-respostas	B → A	struct respostaT, com soma

Fonte: Elaborada pela autora (2026).

Uma única fila não funcionaria, pois pedidos e respostas ficariam misturados e o próprio processo A poderia ler o pedido que acabou de enviar. Os nomes das filas foram personalizados, como orienta o enunciado, para evitar conflitos com as filas de outros alunos.

3.2 IMPLEMENTAÇÃO (ITEM 2)

A solução é composta por quatro arquivos:

- **comum.h**: define os nomes das filas e o formato das mensagens, compartilhados pelos processos A e B;

- **processoA.c**: abre as filas, sorteia n1 e n2, envia o pedido, aguarda a soma e a imprime;
- **processoB.c**: abre as filas, recebe o pedido, calcula a soma, aguarda 1 segundo e envia a resposta;
- **limpa.c**: remove as filas com mq_unlink, para que mensagens de um teste não interfiram no seguinte.

Os dois processos seguem o pseudocódigo do enunciado. O canal de comunicação é aberto uma única vez, antes do laço while (1), por meio de mq_open com a opção O_CREAT; assim, a ordem de inicialização dos processos é indiferente. Cada iteração do laço corresponde a uma rodada completa da comunicação. Como a chamada mq_receive é bloqueante, cada processo permanece suspenso, sem consumir CPU, até a chegada de uma mensagem, o que sincroniza A e B automaticamente (LINUX MAN-PAGES PROJECT, 2026).

Cada processo A inicializa o gerador de números aleatórios com srandom(time(NULL) ^ getpid()). A combinação do horário com o identificador do processo (PID) garante que dois processos A iniciados no mesmo segundo sorteiem números diferentes.

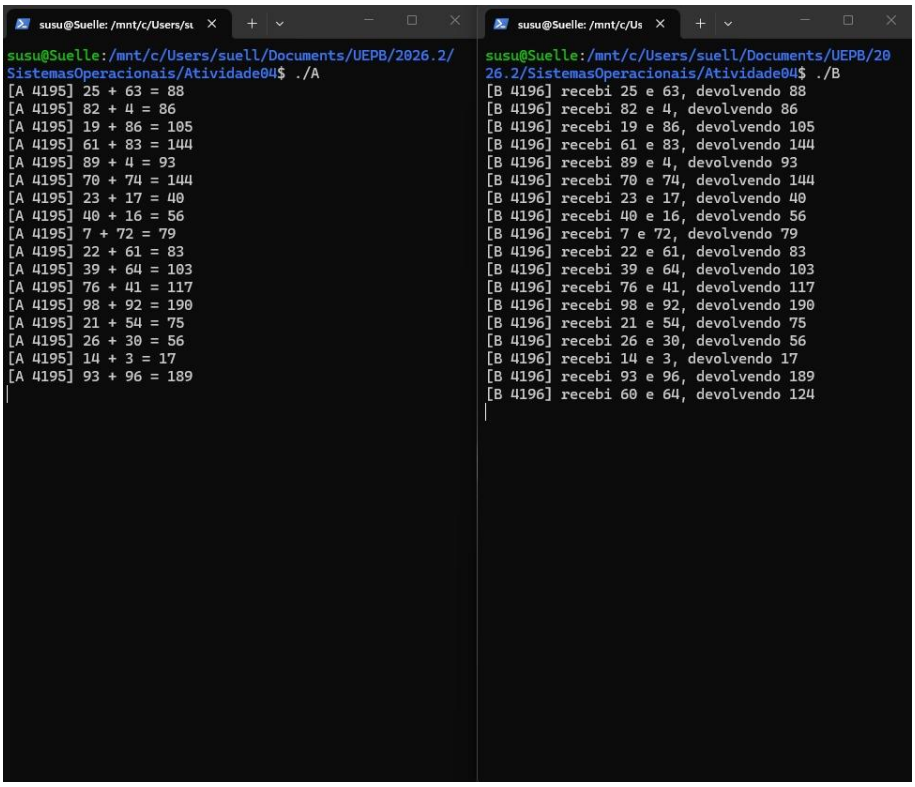
Para facilitar a avaliação, cada linha impressa exibe o PID do processo, e o processo A marca com <-- ERRADO! qualquer soma recebida que não corresponda ao par enviado.

4 AVALIAÇÃO DOS CENÁRIOS

Nas quatro execuções registradas, todas as somas recebidas pelos processos A estavam corretas. Ainda assim, a solução básica só é garantidamente correta com um único processo A, como discutido na seção 5.

4.1 UM PROCESSO A E UM PROCESSO B (ITEM 3)

Figura 1 – Execução com um processo A e um processo B

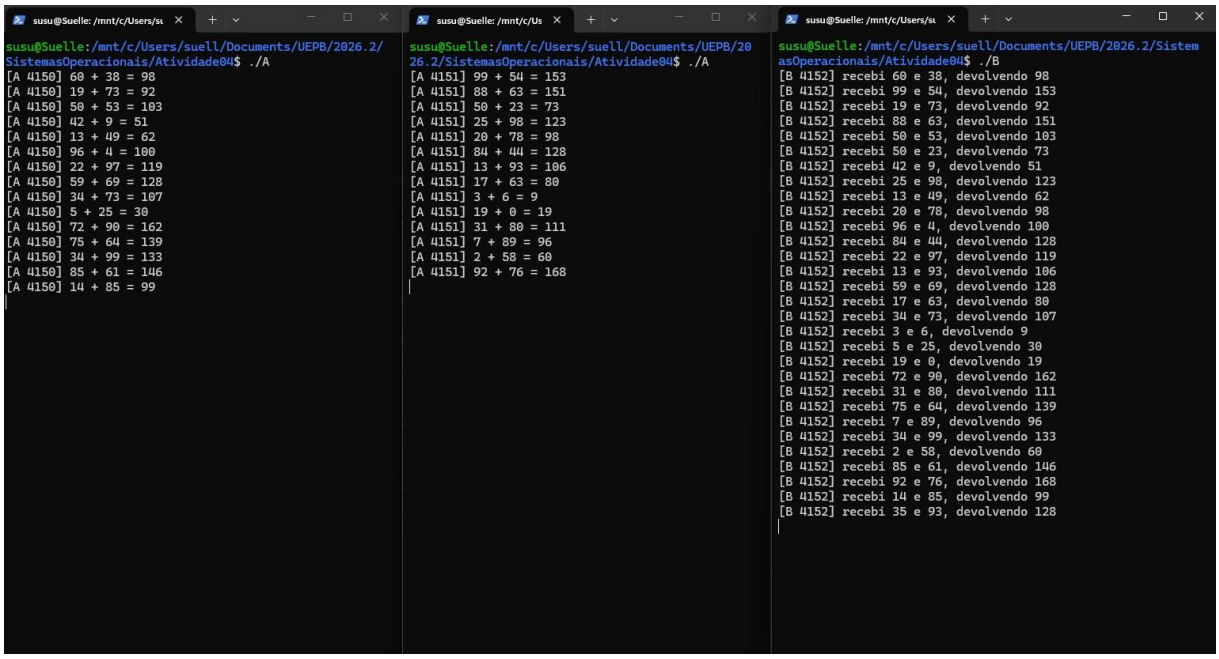


Fonte: Elaborada pela autora (2026).

O funcionamento foi correto: cada pedido do processo A (PID 4195) foi atendido pelo processo B (PID 4196), e todas as somas correspondem aos pares enviados (Figura 1). A última linha do processo B (60 e 64) não aparece no processo A porque este foi encerrado com Ctrl+C antes de receber a resposta. Essa resposta permanece na fila, que persiste no núcleo do sistema após o término dos processos (LINUX MAN-PAGES PROJECT, 2026); por esse motivo, o programa limpa foi executado antes de cada cenário.

4.2 DOIS PROCESSOS A E UM PROCESSO B (ITEM 4)

Figura 2 – Execução com dois processos A e um processo B



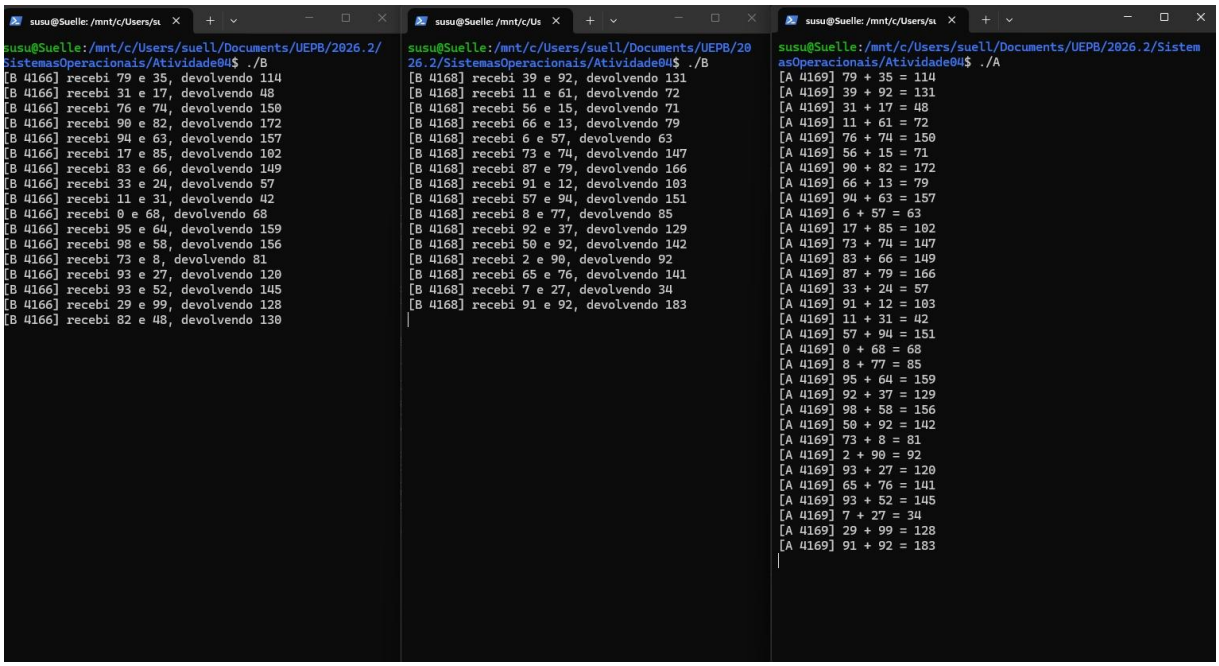
Fonte: Elaborada pela autora (2026).

As somas vieram corretas para os dois processos A (PIDs 4150 e 4151), como mostra a Figura 2. O processo B atende os pedidos por ordem de chegada, alternando entre os dois processos A, à razão de um pedido por segundo. Assim, cada processo A passa a receber uma soma a cada 2 segundos, aproximadamente.

O resultado correto, porém, decorre de uma coincidência. Os dois processos A aguardam na mesma fila de respostas, e a resposta não identifica seu destinatário. A execução funcionou porque o único processo B responde um pedido por vez, em ordem, e o Linux entrega a mensagem ao processo que aguarda há mais tempo, que, neste caso, é o autor do pedido. O padrão POSIX não garante esse comportamento.

4.3 UM PROCESSO A E DOIS PROCESSOS B (ITEM 5)

Figura 3 – Execução com um processo A e dois processos B

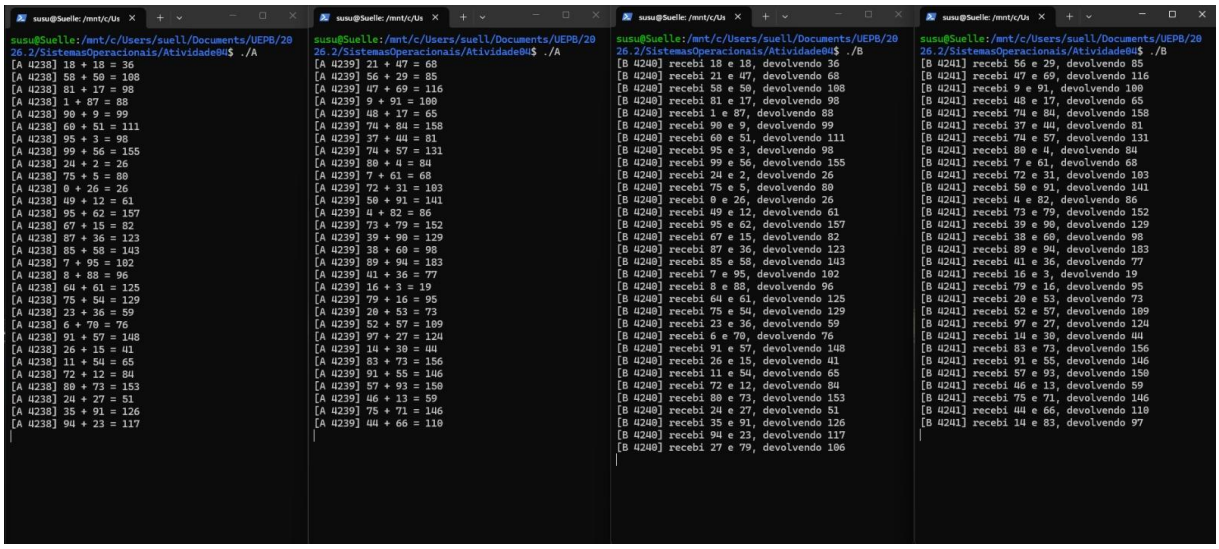


Fonte: Elaborada pela autora (2026).

O funcionamento foi correto, e os pedidos do processo A (PID 4169) foram distribuídos entre os dois processos B (PIDs 4166 e 4168), conforme a Figura 3. Não há, contudo, ganho de desempenho: o processo A só envia um novo pedido após receber a resposta anterior, de modo que há no máximo um pedido em circulação e um dos processos B permanece sempre ocioso.

4.4 DOIS PROCESSOS A E DOIS PROCESSOS B (ITEM 6)

Figura 4 – Execução com dois processos A e dois processos B



Fonte: Elaborada pela autora (2026).

Nesta execução, todas as somas recebidas pelos dois processos A (PIDs 4238 e 4239) estavam corretas (Figura 4). Pelos PIDs, observa-se que os processos se organizaram em pares quase fixos: o processo B 4240 atendeu praticamente todos os pedidos do processo A 4238, e o processo B 4241, os do processo A 4239. Como cada par avançava no mesmo ritmo, cada resposta chegou ao processo que fez o pedido.

Esse resultado também depende da ordem de execução. Com dois processos B trabalhando em paralelo, as respostas podem chegar à fila compartilhada fora de ordem, e um processo A pode receber a soma do pedido do outro. Por isso, este é o cenário em que a solução básica pode falhar, como detalhado a seguir.

5 PROBLEMAS OBSERVADOS E SOLUÇÃO ADEQUADA (ITEM 7)

5.1 O PROBLEMA

A fila de respostas é compartilhada por todos os processos A, e a mensagem de resposta contém apenas a soma, sem indicar o destinatário. Quando há mais de um processo A, cada um aceita a primeira resposta que chegar, que pode corresponder ao pedido de outro processo A. Com dois processos B em paralelo, a ordem das respostas depende de qual deles termina primeiro. Trata-se de uma

condição de corrida, situação em que o resultado depende da ordem em que os processos são executados (MAZIERO, 2019). A sequência a seguir exemplifica como uma soma errada pode ser produzida.

1. A1 envia 4 + 29; B1 recebe o pedido e aguarda 1 segundo.
2. A2 envia 77 + 87; B2 recebe o pedido e aguarda 1 segundo.
3. B2 termina primeiro e deposita 164 na fila de respostas.
4. A1, que aguardava há mais tempo, recebe 164 e imprime 4 + 29 = 164.

Como as execuções registradas não exibiram o erro, conclui-se que o problema é intermitente: ele não se manifesta em todas as execuções, o que o torna difícil de detectar apenas por meio de testes.

Um segundo problema, de menor gravidade, é a persistência das filas no núcleo do sistema após o término dos processos. Mensagens deixadas por um teste interrompido com Ctrl+C interferem no teste seguinte, o que motivou a criação do programa limpa.

5.2 A SOLUÇÃO ADEQUADA

A forma mais adequada de resolver o problema é adotar o modelo **cliente-servidor com uma fila de respostas por cliente**, descrito a seguir:

- a fila de pedidos continua única e compartilhada, de modo que qualquer processo B pode atender qualquer processo A, dividindo a carga entre os servidores;
- cada processo A cria sua própria fila de respostas, com um nome que inclui seu PID, por exemplo /fila-suelle-resp-4238;
- o pedido passa a conter o PID do processo A, além de n1 e n2;
- o processo B abre a fila de respostas correspondente a esse PID e responde apenas nela;
- ao encerrar, cada processo A remove a própria fila com mq_unlink.

Dessa forma, cada processo A lê apenas as respostas destinadas a ele, e o resultado deixa de depender da ordem de execução. Essa versão foi implementada nos arquivos processoA_corrigido.c e processoB_corrigido.c, que acompanham a entrega.

Uma alternativa seria criar uma fila de respostas exclusiva para cada par de processos A e B, mas essa abordagem perderia a vantagem de qualquer processo B poder atender qualquer processo A.

6 CONCLUSÃO

As filas de mensagens POSIX permitiram estabelecer a comunicação entre processos independentes de forma simples, e o caráter bloqueante de mq_receive sincronizou os processos A e B sem a necessidade de mecanismos adicionais. A solução com duas filas funciona corretamente com um único processo A; com vários processos A, porém, passa a depender da ordem de execução, pois as respostas não identificam seu destinatário.

A atividade evidenciou que um sistema concorrente pode passar nos testes e, ainda assim, estar incorreto, e que a correção decorre do projeto da comunicação: atribuir a cada cliente sua própria fila de respostas. Verificou-se também que é possível desenvolver e testar a aplicação no Windows por meio do WSL2, sem o uso de máquina virtual.

REFERÊNCIAS

LINUX MAN-PAGES PROJECT. **mq_overview(7)**: overview of POSIX message queues. Versão 6.19. [*S. l.*], 2026. Disponível em: https://man7.org/linux/man-pages/man7/mq_overview.7.html. Acesso em: 15 out. 2026.

MAZIERO, Carlos Alberto. **Sistemas operacionais**: conceitos e mecanismos. Curitiba: Editora da UFPR, 2019. 456 p. ISBN 978-85-7335-340-2. Disponível em: https://wiki.inf.ufpr.br/maziero/doku.php?id=socm:start. Acesso em: 15 out. 2026.

MICROSOFT. **Como instalar o Linux no Windows com o WSL**. [*S. l.*]: Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/pt-br/windows/wsl/install. Acesso em: 15 out. 2026.